B.TECH PROJECT – PROGRESS REPORT

DevBridge

A Verified-Talent Freelance Platform

Submitted by	*[Name — fill in name / roll number]*
Modules covered	M5 (CV Verification Engine) · M6 (Skill Graph)
Date	September 15, 2026

This report summarises the overall system architecture agreed for DevBridge, the progress made on Module 5 (CV Verification Engine), and the progress made on Module 6 (Skill Graph) – the module this report's author owns.

1 System Architecture and Design Rationale

1.1 The Technical Thesis

DevBridge is a platform for verifying developer competence from evidence rather than self-report. Its central technical claim is:

> **Central claim**
>
> Developer competence can be estimated from heterogeneous, partially reliable behavioural evidence – commit history, contest performance, delivery outcomes, CV claims – by fusing it into a single calibrated **per-skill latent-ability estimate with explicit uncertainty that grows over time**. The fused estimate is more reliable than any single evidence stream, and its stated confidence is empirically accurate.

Everything else in the platform – the marketplace, escrow, the recruiter console – is supporting infrastructure. It is competently engineered, but it is the skill-graph model and the verification pipeline that constitute the project's technical contribution.

1.2 Architectural Principles

Six rules, agreed before any module's implementation began, from which every later design decision follows.

Principle	What it means in practice
Contracts before code	Every module publishes a schema/OpenAPI contract before implementation starts. A module is "done" when it satisfies its contract tests.
No shared database	Every module owns its storage exclusively. Cross-module communication happens over HTTP or events, never a shared table.
Evidence is append-only	Nothing overwrites a record. Every score is *derived and recomputable* from the event log at any point in time.
The LLM boundary	A language model may translate unstructured text into structured data, or structured findings into prose. It may **never** produce a number that enters a score, and never decide whether a claim is true.
No trained models	Every quantitative judgement comes from descriptive statistics against a reference distribution or a closed-form update rule – never a fitted classifier. There is no labelled training set anywhere in the pipeline.
Confidence, never verdicts	Every module emits a value with confidence and evidence references. No module emits a boolean pass/fail or a hiring recommendation.

1.3 Module Map

The platform is organised into fifteen independently deployable modules across five layers. M5 and M6 – the subjects of this report – are highlighted.

Layer	Module	Responsibility
0 – Substrate	M0 core-gateway	Edge routing, authentication, rate limiting
	M1 identity-consent	Accounts and an append-only consent ledger
	M2 evidence-bus	Event backbone and schema registry
1 – Evidence producers	M3 code-provenance	Authorship and originality verification from repository history
	M4 assessment-engine	Sandboxed technical assessments
	M5 cv-verification	**The standalone CV verification engine – §2**
2 – The graph	**M6 skill-graph**	**Fuses all evidence into calibrated per-skill ability – §3**
3 – Marketplace	M7 marketplace	Postings, matching, contract state machine
	M8 delivery-review	Post-gig review; feeds evidence back to M6
	M9 payments-escrow	Escrow ledger and payout
	M10 financing	Advance-rate decisioning, priced off the skill graph
4 – Enterprise	M11 enterprise-api	Public verification API, metering
	M12 ats-connector	ATS integration adapters
	M13 recruiter-console	Consent-gated search over the verified pool
Cross-cutting	M14 eval-harness	Datasets, metrics, ablation, calibration
	M15 web-clients	Front-ends against the public contracts only

1.4 Block Diagram

Evidence flows one way, bottom to top: producer modules generate typed evidence, the skill graph fuses it into calibrated ability, and the marketplace and enterprise layers read from that fused state. No module reads another's private database.

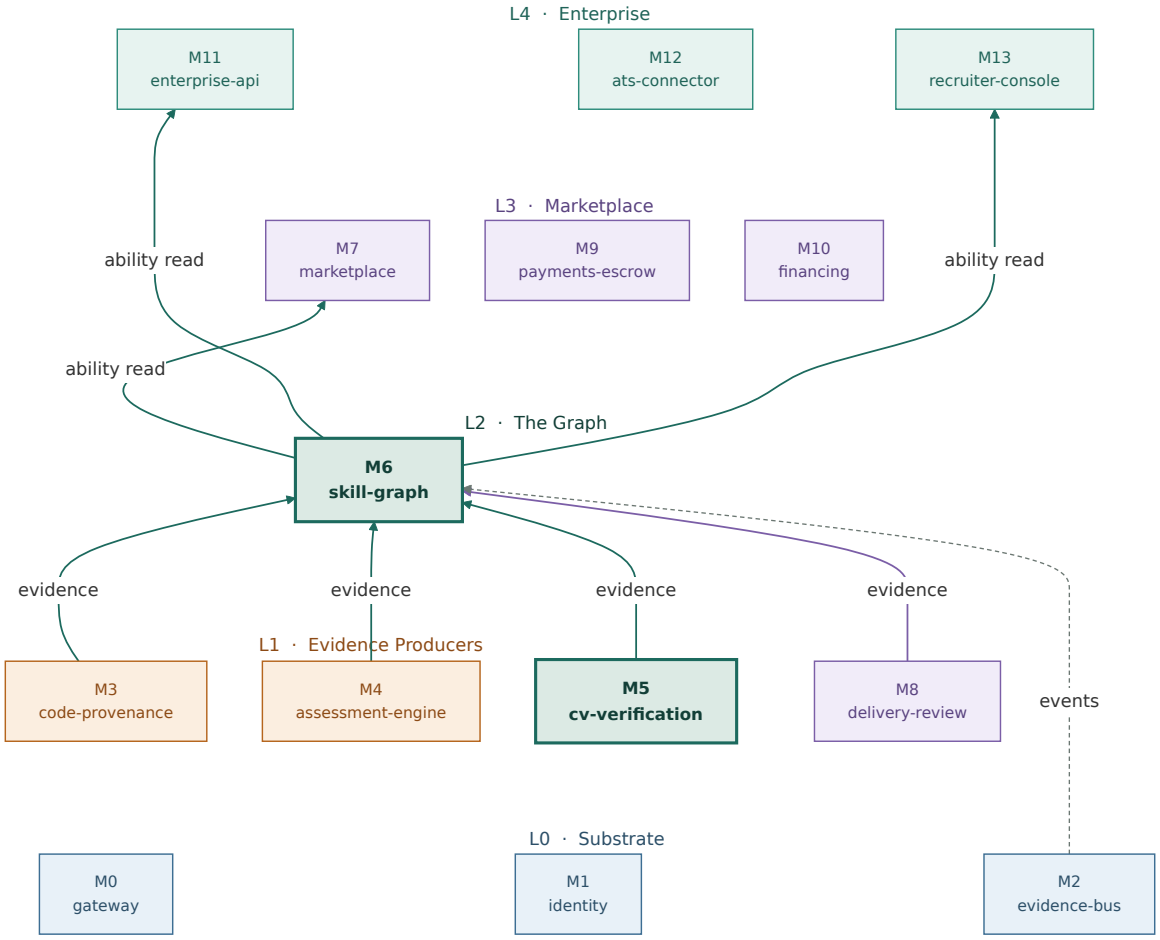


Figure 1: DevBridge module map. M5 and M6 are shaded. Arrows show the direction evidence and ability estimates flow; no module reads another's private storage.

1.5 Where Language Models Are Used, and Where They Are Not

A model is used only where no deterministic alternative exists: extracting structured claims from free-text CVs (M5), generating novel assessment challenges (M4), and rendering computed findings into prose (M5, M6-adjacent reporting). A model is explicitly **forbidden** from computing any score, judging authorship, deciding timeline consistency, or issuing any verdict – those are deterministic rule engines, statistical distance measures, or closed-form filters, precisely so that every number in the system can be recomputed and audited without re-running a model.

2 Module 5 — CV Verification Engine

2.1 Purpose and Constraints

M5 turns an uploaded CV into a structured report of what the evidence supports, what it contradicts, and what could not be checked. It is built as a **standalone product**: it runs on any CV, including one from a person with no DevBridge account; it imports nothing from the rest of the platform; and it emits findings, never a verdict. There is no overall trust score, no threshold, and no pass/fail – a design constraint enforced in code, not only stated in documentation.

2.2 The Pipeline

The pipeline runs in nine steps. Steps 2, 3 and 9 call a language model; every other step is deterministic code with no network dependency beyond the evidence lookups in step 6.



Figure 2: The nine-step verification pipeline. A quote that cannot be found in the source document is dropped at step 4; a real but unusable claim is refused at step 5; every rule outcome at step 7 is one of `discrepancy`, `corroborated`, `abstain` or `clean` – never a verdict.

Step	What happens
1 Read file	PDF/DOCX/OCR extraction to normalised text with page offsets.
2–3 Extract	The model proposes claims (employment, education, skills, projects, achievements) with a verbatim quoted span – no character positions are trusted from the model.
4 Ground	Every quoted span must literally exist in the source document (exact or a fuzzy match ≥ 0.92); a claim that fails is dropped and counted.
5 Filter	A real but unusable claim (value not supported by its own sentence, duplicate, excluded exam) is refused.
6 Gather evidence	GitHub, Codeforces, LeetCode and live-URL lookups, only for accounts the CV itself names. A failed lookup becomes a typed gap, never a guess.
7 Run the checks	27 pure-function rules, each producing a `discrepancy`, an `abstain` with a reason, a `corroborated`, or a `clean`.
8 Sort	Findings sort into five buckets by the rule's own tier – *Contradicted*, *Needs context*, *Verified*, *Not evaluated*, *Out of scope* – never into a single score.
9 Hand it on	The same report as JSON/API, a one-sentence-per-finding narrative, a reviewer worksheet, and (with consent) skill signals to M6.

2.3 Progress to Date

Metric	Value	Note
Checks built	28 / 34 reachable	6 remaining need issuer APIs DevBridge does not yet have
Automated tests	474 passing, 0 failing	plus 5/5 on the live-model integration suite
Static analysis	clean	`ruff` and `mypy -strict` across the codebase
Source size	∼9,960 lines	across the pipeline, rule engine and API
Documents evaluated	3 real CVs	student, synthetic, and professional profiles
Injected-error detection	15/15, 7/7, 2/2	0 false positives across all three documents
Span-grounding failures	0	every dropped claim was refused text, never fabricated

Check family	Built	Basis	Status of the rest
Document-intrinsic	11 / 11	the CV alone (timeline, tenure, dates)	complete
GitHub	10 / 10	the account and repositories named	complete
Contest platforms	5 / 6	Codeforces, LeetCode	CodeChef has no public API
Deployments	1 / 2	does the linked site respond	"matches description" needs a reviewer
Credentials	1 / 5	narrative rendering only	no issuer APIs available yet
Human-judgement	0 / 6	*by design*	prepared as a worksheet, never automated

The pipeline is feature-complete for everything currently reachable without an issuer integration. The two items of remaining work are a larger hand-labelled evaluation corpus (to measure extraction precision/recall directly, alongside the detection-rate figures above) and issuer-API integrations for the credential family.

3 Module 6 — Skill Graph

3.1 Purpose

M6 takes verified evidence about a developer – from M5, M3, M4 and delivery history – and maintains a calibrated belief about that developer's ability in each individual skill: not a rating, but a Gaussian estimate that narrows as evidence arrives and widens again as it ages. There is no overall score; every answer is per skill, with an explicit interval.

3.2 Module Architecture

Six submodules, evidence flowing one way. The ontology is consulted, never written to, by three of the six; the integrity layer reads the finished graph and writes nothing back into it.

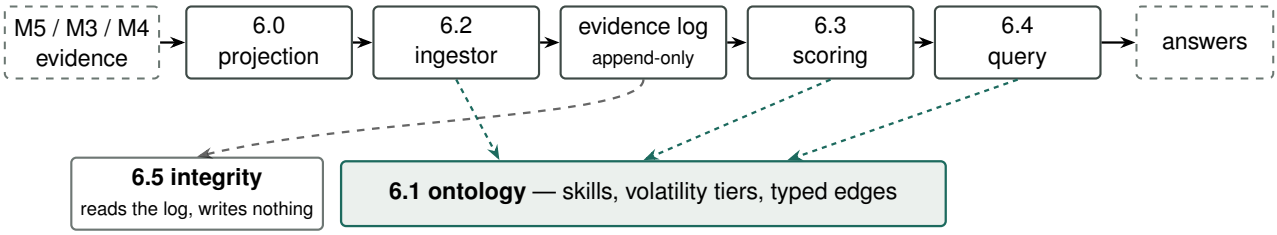


Figure 3: The six M6 submodules. 6.1 exposes three read-only questions (`knows()`, `tier()`, `relations()`); nothing downstream writes back to it. 6.5 changes no score it reads.

3.3 The Scoring Model

Each (developer, skill) pair is a Gaussian belief in **logit space**: $x = \text{logit}(\theta)$, unbounded, so the Kalman update below is exact rather than an approximation later clipped to $[0, 1]$.

Observation. Every event yields an observed value and a variance:

$$\sigma^2_{\text{obs}} = \frac{S_0}{\text{reliability} \times \text{relevance} \times \text{strength}}, \qquad S_0 = 0.10.$$

Relevance sits in the variance, never the observed value – a signal that is only partially about a skill is treated as *less certain*, not as *weaker evidence of a lower ability*.

Update. A closed-form Kalman step, no fitting:

$$K = \frac{\sigma^2}{\sigma^2 + \sigma^2_{\text{obs}}}, \qquad \mu' = \mu + K(\text{logit}(\text{observed}) - \mu), \qquad \sigma^{2\prime} = \sigma^2(1 - K).$$

Decay. Between observations, uncertainty grows and the mean reverts toward the prior:

$$\sigma^2(t) = \sigma^2(\tau) + \lambda_s \, \Delta t, \qquad \mu(t) = \mu + (\mu_0 - \mu)\big(1 - e^{-\Delta t \, \lambda_s / k}\big).$$

λ_s depends on the skill's volatility tier (fundamental, ecosystem, framework) – a framework goes stale in about 1.3 years, a fundamental in about 9.6.

Display. The published number is a **lower confidence bound**, not the mean:

$$\text{score} = \text{sigmoid}(\mu - z\sigma), \qquad z = 1.0.$$

Thin evidence and abundant evidence can share a mean; only the second produces a high published score, because only its interval has closed up.

3.4 Worked Example

The table below is a real trace from the shipped scoring engine for one developer's `go` skill: a repository commit, a corroborated CV claim, and 81 days of subsequent silence.

Step	μ	σ	score	mean
prior (no evidence)	-1.099	1.300	0.083	0.250
+ repository commit (168 commits)	$+0.436$	0.467	0.492	0.607
+ corroborated CV claim	$+0.611$	0.313	0.574	0.648
81 days later, published	$+0.496$	0.350	**0.536**	0.622

The published score rises from 0.083 to 0.574 mainly because the *interval closes* (width $0.47 \rightarrow 0.14$), not because the mean moves furthest. It then falls to 0.536 purely from elapsed time – the system stating it knows less, never that the developer regressed.

3.5 Progress to Date

Seven packages, all independently tested.

Submodule	Package	Lines	Tests
Facade	`m6`	316	37
6.1 ontology	`m6_ontology`	411	77
6.0 projection	`m6_projection`	904	108
6.2 ingestor	`m6_ingestor`	711	76
6.3 scoring	`m6_scoring`	1,019	129
6.4 query	`m6_query`	618	97
6.5 integrity	`m6_integrity`	412	38
Total		**4,391**	**562**

Validation. A parameter-recovery experiment – 300 synthetic developers of known ability, 12 events each, folded through the shipped code – gives bias -0.0236 (target < 0.05), RMSE 0.0510 (target < 0.15), and one-sigma coverage of 0.677 against a nominal 0.682: the filter's stated uncertainty matches its actual uncertainty, with no parameter tuned to produce that result.

Remaining work. The platform-rating reference distribution (Codeforces percentile table) is currently an approximation pending regeneration from the live user list; the full parameter sensitivity sweep is implemented but not yet run end to end and written up.